

Lab 04: Clustering and Association Rule Learning

July 02, 2026

Objectives

To provide students hands-on experience in clustering and Association Rule Learning using Python. At the end of the lab, students should be able to:

- Use the K-Means algorithm for a given dataset.
- Analyze the output of the K-Means algorithm and interpret the results.
- Use the Apriori algorithm to discover association rules.
- Analyze the output of the Apriori algorithm and interpret the results.

1 Clustering

1.1 Importing Required Modules

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs # to generate sample datasets
```

1.2 Creating a Sample Dataset with 4 Clusters

```
X, y = make_blobs(
    n_samples=400,
    centers=4,
    cluster_std=0.90,
    random_state=1
)
```

1.3 Determining the Optimum Value of k Using the Elbow Method

```
import matplotlib.pyplot as plt

wcscs = [] # within cluster sum of squares
for i in range(1, 11):
    kmeans = KMeans(
        n_clusters=i,
        init='k-means++',
        max_iter=300,
        n_init=10,
        random_state=0
    )
    kmeans.fit(X)
    wcscs.append(kmeans.inertia_)

plt.plot(range(1, 11), wcscs)
plt.title('Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```

1.4 Applying the K-Means Algorithm

```
kmeans = KMeans(n_clusters=4, random_state=0) # from Elbow method
closest_cluster_index = kmeans.fit_predict(X)
cluster_centers = kmeans.cluster_centers_
```

1.5 Visualizing Clusters

```
plt.scatter(X[:, 0], X[:, 1], c='green')
plt.scatter(
    cluster_centers[:, 0],
    cluster_centers[:, 1],
    s=200,
    c='black',
    marker='*'
)
plt.show()
```

Exercise 01

1. Import the iris dataset from scikit-learn. Convert it into an unlabeled dataset by removing the class attribute.
2. Use the Elbow method to identify the best value for k (minimizing WCSS).
3. Fit the K-Means algorithm with the k found in part (b).
4. Explain the output of:

```
kmeans.cluster_centers_
```

where `kmeans` is your fitted `KMeans` object.

5. Visualize the data points and cluster centers in a 3D plot using the first three features as axes.

6. Compute the `silhouette_score` for $k = 2$ to 10. Does the best k agree with the Elbow Method?
7. Compute the `adjusted_rand_score` between your K-Means labels and `iris.target`. Why is this check possible here but not for typical unlabeled data?
8. Run `KMeans` twice with `n_init=1`, using two different `random_state` values. Compare the `cluster_centers_` and `inertia_`. Why can they differ?
9. Generate a `make_moons` dataset (400 samples, `noise=0.07`). Cluster it with `KMeans(n_clusters=2)` and with `DBSCAN`, plot the two results side by side, and explain why K-Means fails here but DBSCAN doesn't.

2 Association Rule Learning

2.1 Installing Apriori

```
pip install mlxtend # installs the mlxtend library
```

2.2 Importing Required Modules

```
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
from mlxtend.preprocessing import TransactionEncoder
import pandas as pd
```

2.3 Input Data

```
dataset = [
    ['Milk', 'Onion', 'Nutmeg', 'Kidney Beans', 'Eggs', 'Yogurt'],
    ['Dill', 'Onion', 'Nutmeg', 'Kidney Beans', 'Eggs', 'Yogurt'],
    ['Milk', 'Apple', 'Kidney Beans', 'Eggs'],
    ['Milk', 'Unicorn', 'Corn', 'Kidney Beans', 'Yogurt'],
    ['Corn', 'Onion', 'Onion', 'Kidney Beans', 'Ice cream', 'Eggs']
]
```

2.4 Creating the DataFrame of Frequent Itemsets

```
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)
```

2.5 Applying the Apriori Algorithm and Finding Association Rules

```
freq = apriori(df, min_support=0.002, use_colnames=True)
rules = association_rules(freq, metric="lift", min_threshold=1)
```

Exercise 02

1. Import the provided `groceries.csv` dataset.
2. Explore the dataset and build the frequent-item DataFrame.
3. Apply the Apriori algorithm to find itemsets with support $> 8\%$.
4. Generate association rules using the lift metric.
5. Select one rule and interpret it in your own words.
6. How many rules satisfy both lift > 4 and confidence > 0.8 ?
7. Pick one rule and manually compute its support, confidence, and lift from the raw transactions. Confirm it matches `association_rules()`.
8. Does $\text{lift}(A \rightarrow B) = \text{lift}(B \rightarrow A)$? Work it out algebraically, then check numerically on one rule pair.
9. Find a high-support/low-confidence rule and a high-lift/low-support rule. What does each combination tell you about trusting the rule?
10. Re-run itemset mining on `groceries.csv` using `fpgrowth` with the same `min_support`. Compare the itemsets found and the runtime against Apriori, and explain why FP-Growth is usually faster.

Submission

Submit:

- Two Python scripts—one for the clustering exercise and one for association-rule learning.
- A single PDF containing answers for both Exercises 01 and 02.

Place all files in a folder, compress it, and name it `e21xxxlab4`, where `xxx` is your registration number.